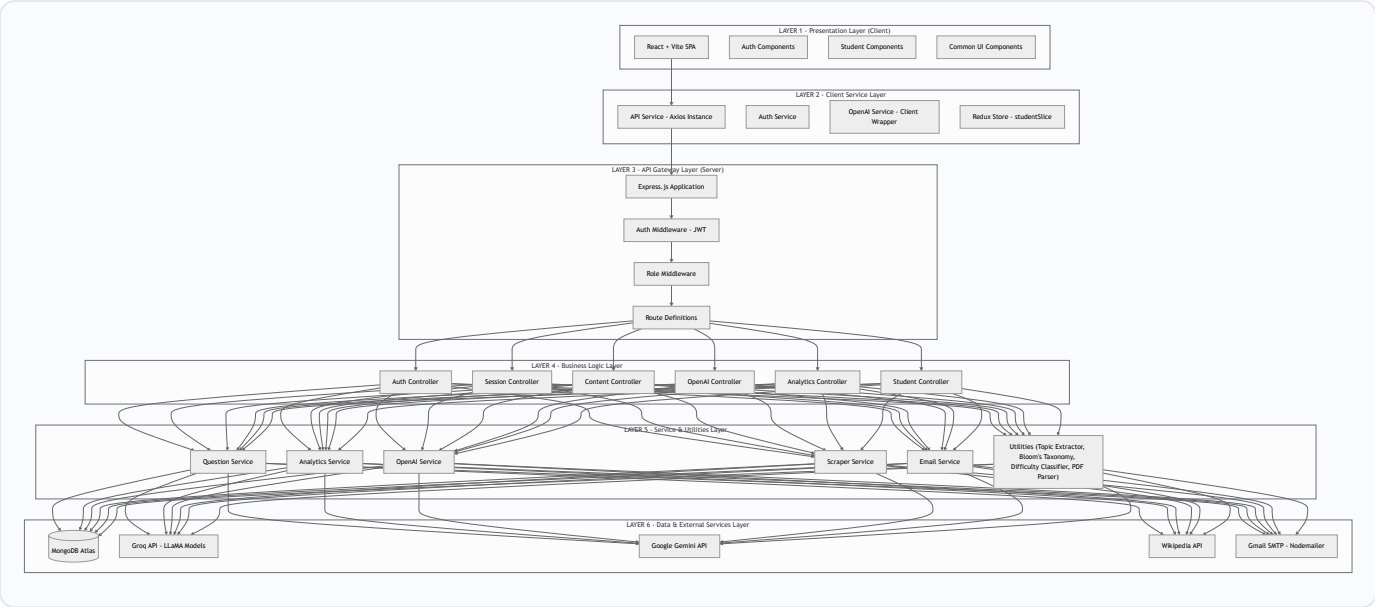
DLP System Architecture and Data Flow Report

Digital Learning Platform — System Architecture & Data Flow Report

> **Document Purpose:** This report presents a comprehensive, layer-by-layer decomposition of the Digital Learning Platform (DLP) architecture, accompanied by an exhaustive analysis of data flow from user authentication through examination analysis. It is intended to serve as the definitive reference for constructing High-Level System Design diagrams and Data Flow Diagrams (DFDs).

1. High-Level Architecture Overview

The DLP follows a **three-tier client-server architecture** with a decoupled frontend and backend, communicating exclusively via RESTful JSON APIs over HTTP. The system is further decomposed into **six logical layers**.



2. Layer-by-Layer Decomposition

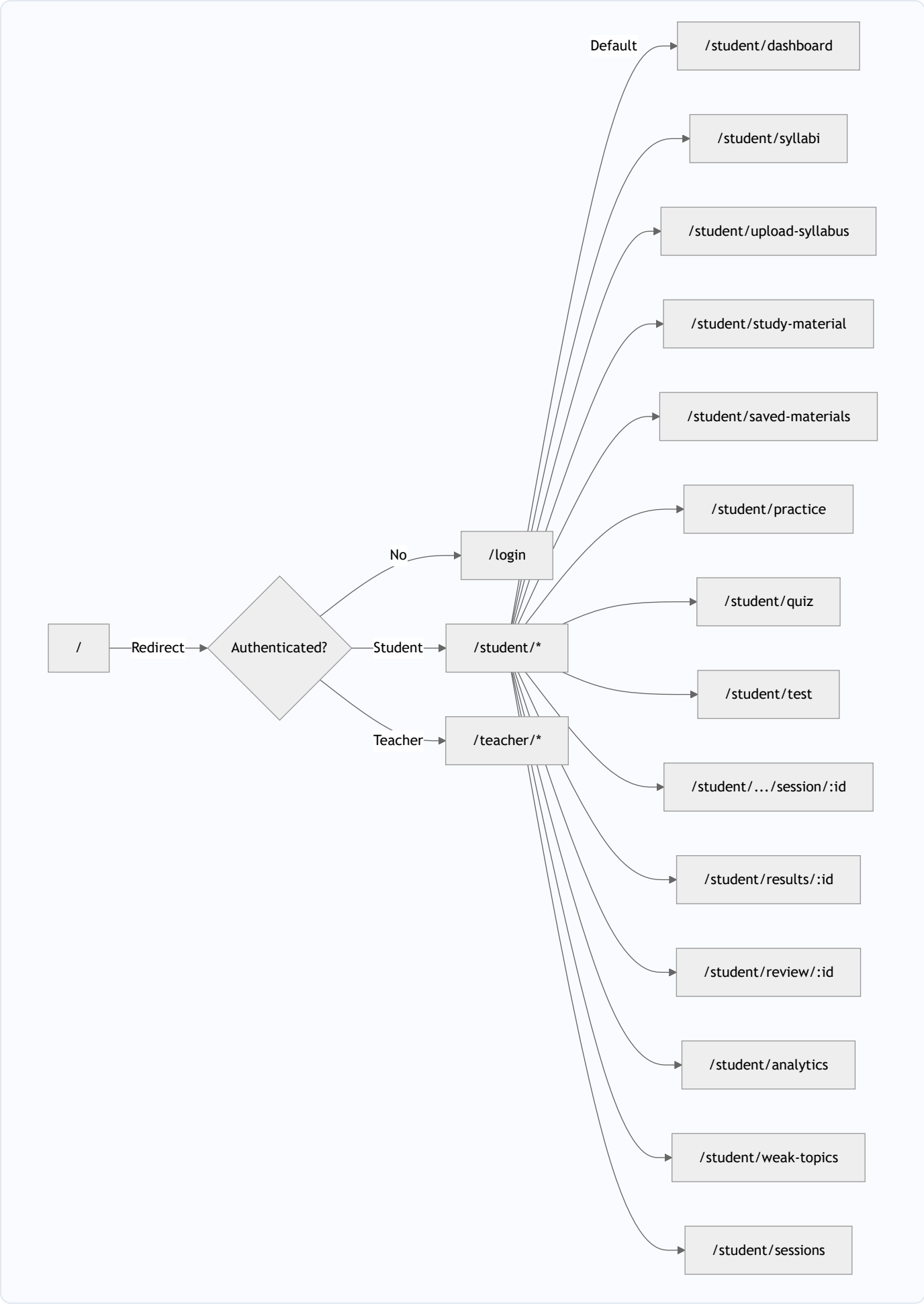
2.1 LAYER 1 — Presentation Layer

Aspect	Detail
Technology	React 18 + Vite (JSX, CSS Modules)
State Management	Redux Toolkit (`@reduxjs/toolkit`) via `studentSlice`
Routing	React Router v6 with nested route definitions
Styling	Vanilla CSS with component-scoped stylesheets

2.1.1 Components Taxonomy

Component Group	Key Components	Responsibility
Auth	`LoginForm` , `RegisterForm`	Credential collection, form validation, login/register API dispatch
Student	`Dashboard` , `SyllabusManager` , `UploadSyllabus` , `StudyMaterial` , `SavedMaterials` , `ViewMaterial` , `PracticeSession` , `QuizSession` , `TestSession` , `QuestionView` , `ResultPage` , `SessionReview` , `Analytics` , `WeakTopics` , `SessionsList`	Full student lifecycle — from content consumption to examination review
Common	`Navbar` , `Sidebar` , `Footer` , `CustomDropdown` , `Icons`	Shared UI chrome and reusable interaction elements
Pages	`Login` , `Register` , `ForgotPassword` , `ResetPassword` , `Profile` , `StudentHome` , `TeacherHome`	Top-level page wrappers that compose components

2.1.2 Routing Architecture


/
Redirect
Authenticated?
No
/login
Student
/student/*
Teacher
/teacher/*
Default
/student/dashboard
/student/syllabi
/student/upload-syllabus
/student/study-material
/student/saved-materials
/student/practice
/student/quiz
/student/test
/student/.../session/:id
/student/results/:id
/student/review/:id
/student/analytics
/student/weak-topics
/student/sessions

Key Activity: The `ProtectedStudentRoute` wrapper verifies authentication (JWT in `localStorage`) and role (`student`) before rendering any child route. Unauthorized access triggers a redirect to `/login`.

2.2 LAYER 2 — Client Service Layer

This layer encapsulates all HTTP communication and client-side state management. It acts as the **single point of contact** between the UI and the server.

2.2.1 API Service (api.js)

Feature	Implementation
HTTP Client	Axios instance with base URL from `VITE_API_BASE_URL`
Request Interceptor	Automatically injects `Authorization: Bearer <token>` from `localStorage` on every request
Response Interceptor	Catches `401 Unauthorized` responses → clears token → redirects to `/login`
Endpoints Map	Centralized `endpoints` object defining all API paths for auth, student, openai, and RAG operations

2.2.2 Auth Service (auth.js)

Provides `login()`, `register()`, `logout()`, `getCurrentUser()`, `isAuthenticated()`, `getUserRole()`, `forgotPassword()`, `resetPassword()`, and `editPassword()`. Includes a dormant **Mock Mode** for offline testing.

Key Activity: On successful login/register, the JWT token and user object are persisted to `localStorage`, establishing the client-side authentication state.

2.2.3 OpenAI Service (openai.js)

Wraps all domain-specific API calls: `getDashboard()`, `getSyllabi()`, `uploadSyllabus()`, `generateMaterial()`, `generateComprehensiveMaterial()`, `startSession()`, `submitAnswer()`, `completeSession()`, `getSessionResult()`, `getSessionQuestions()`, `getAnalytics()`, `getWeakTopics()`, `getProgressHistory()`, `getTopicPerformance()`, `getSessionHistory()`.

2.2.4 Redux Store (studentSlice.js)

Async Thunk	Data Managed	Source Endpoint
`fetchDashboard`	Dashboard stats, recent syllabi, recent sessions, weak topics	`GET /api/student/dashboard`
`fetchSyllabi`	Array of user syllabi	`GET /api/student/syllabi`
`fetchAnalytics`	Cumulative analytics object	`GET /api/student/analytics`
`fetchWeakTopics`	Array of weak topic objects	`GET /api/student/analytics/weak-topics`
`fetchSessionHistory`	Paginated session list	`GET /api/student/sessions`

Synchronous Reducers: `setCurrentSession`, `clearCurrentSession`, `updateSessionAnswer`, `clearErrors` — manage in-memory session state during active examination.

2.3 LAYER 3 — API Gateway Layer

Aspect	Detail
Framework	Express.js (ESM modules)
Entry Point	`server.js` → connects MongoDB → starts HTTP listener
Middleware Stack	`morgan` (logging) → `express.json` (10MB limit) → `cors` (client origin)

2.3.1 Route Groups

Route Prefix	Auth Required	Role Required	Controller(s) Served
`/api/auth`	Partial (login/register are public)	None	`authController`
`/api/student`	Yes	`student`	`studentController`, `contentController`, `sessionController`, `analyticsController`
`/api/teacher`	Yes	`teacher`	Placeholder handlers (Phase 2)
`/api/openai`	Yes	None	`openaiController`

2.3.2 Authentication Middleware (authMiddleware.js)

Activity: 1. Extracts the `Authorization` header. 2. Splits `Bearer <token>` and validates the token against `JWT_SECRET` using `jsonwebtoken.verify()`. 3. Decodes the payload `{ userId, role }` and attaches it to `req.user`. 4. Rejects with `401` if the token is missing, malformed, or expired.

2.3.3 Role Middleware (roleMiddleware.js)

Activity: Compares `req.user.role` against the required role. Returns `403 Forbidden` if the role does not match.

2.4 LAYER 4 — Business Logic Layer (Controllers)

2.4.1 Auth Controller (authController.js)

Function	Data Flow
`register(req, res)`	Receives `{name, email, password, role}` → validates uniqueness → hashes password (bcrypt, 10 rounds) → creates `User` document → creates initial `Analytics` document → signs JWT (7-day expiry) → returns `{token, user}`
`login(req, res)`	Receives `{email, password}` → finds user by email OR username → verifies password via `bcrypt.compare()` → signs JWT → returns `{token, user}`
`forgotPassword(req, res)`	Generates `crypto.randomBytes(32)` reset token → hashes with SHA-256 → stores hash + 30-min expiry in `User` → constructs reset URL → dispatches email via `emailService`
`resetPassword(req, res)`	Receives `{token, newPassword}` → hashes incoming token → matches against DB → updates `passwordHash` → clears reset fields
`editPassword(req, res)`	Requires current JWT → verifies old password → hashes and saves new password
`getProfile(req, res)`	Returns `User` document excluding `passwordHash`
`updateProfile(req, res)`	Accepts all profile fields → validates username/email uniqueness → constructs `name` from parts → performs `findByIdAndUpdate`

2.4.2 Student Controller (studentController.js)

Function	Data Flow
`getDashboard`	Aggregates: user info, 5 recent syllabi, 5 recent sessions, analytics summary (with login streak update), total counts, weak topics
`getSyllabi`	Returns all syllabi for the authenticated user, sorted by creation date
`getSyllabusById`	Returns a single syllabus by ID, ownership-verified
`updateSyllabus`	Updates topics and optionally fileName for a syllabus
`getSessionHistory`	Returns paginated session history with optional type filter, populating linked syllabus fileName

2.4.3 Content Controller (contentController.js)

Function	Data Flow
`uploadSyllabus`	Receives `{fileName, content, pdfBase64}` → if PDF, parses via `pdfParser` → creates `Syllabus` (status: pending) → invokes `extractTopics()` AI utility → updates topics + status
`generateStudyMaterial`	Receives `{syllabusId, topic, mode}` → verifies ownership → calls `generateStudyContent()` service → if mode ≠ "short", auto-saves to `StudyMaterial` collection
`saveStudyMaterial`	Explicitly saves material with custom name to `StudyMaterial`
`combineMaterials`	Merges multiple `StudyMaterial` documents into a single combined material with concatenated content/sections
`getAllStudyMaterials`	Returns paginated, searchable, filterable list of user's saved materials
`deleteStudyMaterial` / `deleteSyllabus`	Ownership-verified deletion

2.4.4 OpenAI Controller (openaiController.js)

Function	Data Flow
`generateMaterial`	**Short mode:** Single AI call → returns JSON sections. **Intermediate/Pro mode:** 5 parallel AI calls (one per section: Introduction, Key Concepts, In-Depth, Applications, Summary) → merges results
`generateComprehensiveMaterial`	Per-subtopic parallel generation. First classifies subtopics as code-intensive or not (single batched AI call). Then for each subtopic: generates concept explanation + code examples (split into 2 API calls for Pro/Intermediate to avoid token limits) + optional advanced insights for Pro mode. Merges all results.
`generateQuestionsAI`	Direct AI question generation (MCQ format) for a topic
`parseSyllabus`	AI-powered syllabus parsing into structured topics/subtopics
`analyzeWeakness`	AI-powered analysis of student performance data with recommendations

2.4.5 Session Controller (sessionController.js)

Function	Data Flow
`startSession`	Receives `{type, syllabusId, topics, difficulty, questionCount, timeLimit, questionMode, sectionsConfig}` → invokes `generateQuestions()` service → de-duplicates → creates `Session` document with embedded question snapshots → returns session with questions (no correct answers)
`submitAnswer`	Receives `{questionId, answer, timeSpent}` → fetches `Question` → if descriptive: AI evaluation via `evaluateDescriptiveAnswer()` → if MCQ: string comparison → stores answer in session. In practice mode: returns correctness + explanation immediately
`completeSession`	Calculates: total/obtained marks, difficulty analysis, topic performance, time analytics → creates `Result` document → updates `Session` status → invokes `updateUserAnalytics()`
`getSession`	Returns full session document (ownership-verified)
`getSessionResult`	Returns `Result` document, populating session metadata
`getSessionQuestions`	Returns all questions with user answers, correct answers, and explanations for post-session review

2.4.6 Analytics Controller (analyticsController.js)

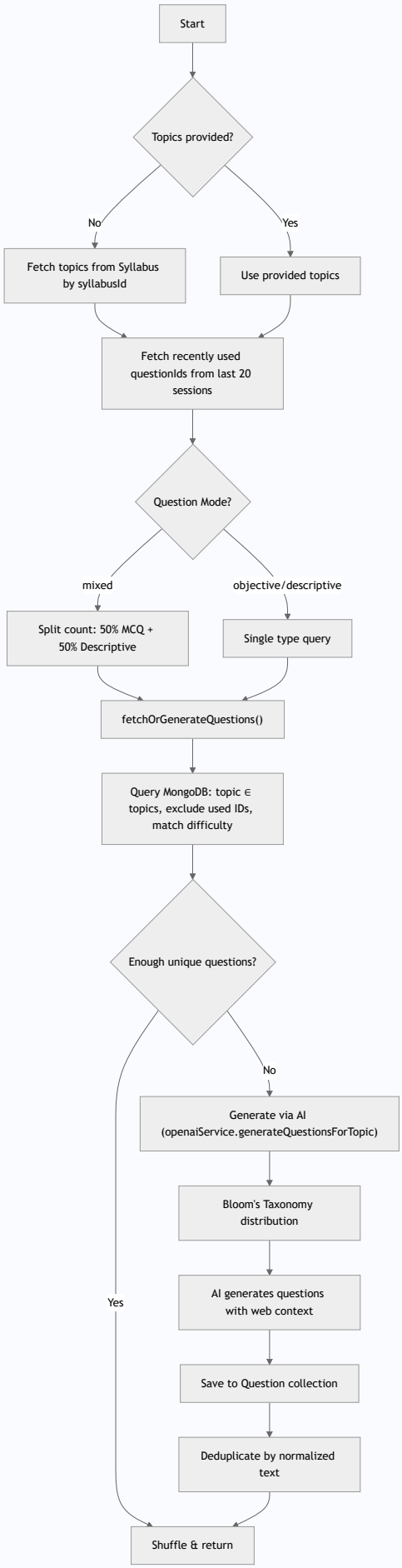
Function	Data Flow
`getAnalytics`	Returns the user's cumulative `Analytics` document
`getWeakTopics`	Filters `Analytics.weakTopics` where accuracy < 50%, sorted by improvement priority
`getProgressHistory`	Queries `Result` documents from the last N days → groups by date → computes daily averages. Returns both server-grouped and raw data for client-side local grouping
`getTopicPerformance`	Aggregates topic performance across ALL `Result` documents for the user
`getDifficultyPerformance`	Returns difficulty breakdown from the cumulative `Analytics` document

---

2.5 LAYER 5 — Service & Utilities Layer

2.5.1 Question Service (questionService.js)

Core Function: `generateQuestions()`


Start
Topics provided?
No
Yes
Fetch topics from Syllabus by syllabusId
Use provided topics
Fetch recently used questionIds from last 20 sessions
Question Mode?
mixed
objective/descriptive
Split count: 50% MCQ + 50% Descriptive
Single type query
fetchOrGenerateQuestions()
Query MongoDB: topic ∈ topics, exclude used IDs, match difficulty
Enough unique questions?
No
Yes
Generate via AI (openaiService.generateQuestionsForTopic)
Bloom's Taxonomy distribution
AI generates questions with web context
Save to Question collection
Deduplicate by normalized text
Shuffle & return

Key Activities: - **Deduplication:** Three-layer dedup — (1) normalized text during DB fetch, (2) normalized text during AI generation, (3) final safety-net dedup before return. - **Bloom's Taxonomy:** Questions are distributed across 6 cognitive levels (Remember → Create) based on difficulty weighting. - **Marks Assignment:** Easy=1, Medium=2, Hard=3.

2.5.2 Analytics Service (analyticsService.js)

Core Function: `updateUserAnalytics(userId, result)`

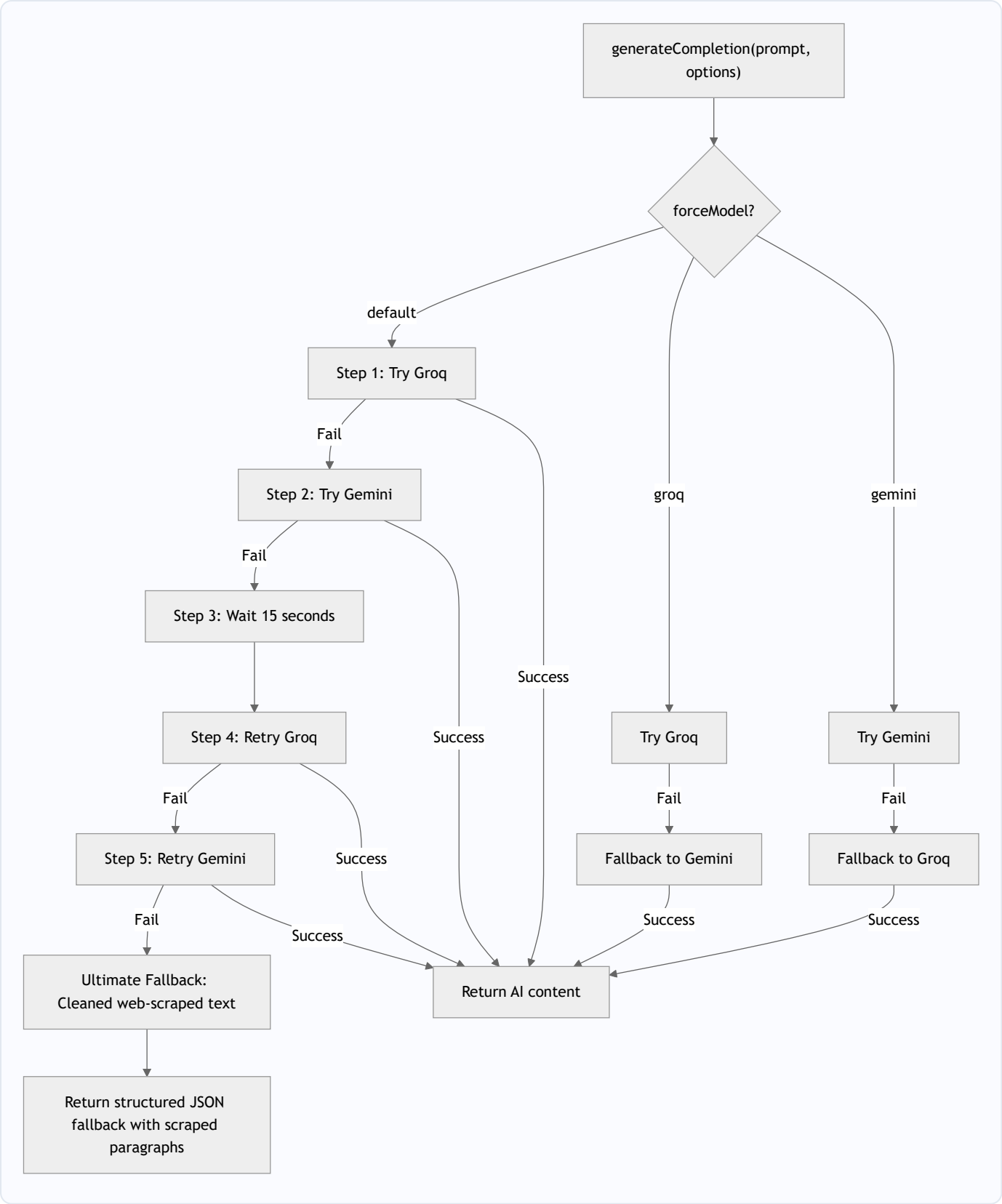
Activities: 1. Increments `totalSessions` and `totalQuestionsAttempted`. 2. Recalculates `overallAccuracy` as a running average. 3. Updates `avgTimePerQuestion` as a rolling mean. 4. Updates `difficultyPerformance` (easy/medium/hard) with new attempted/correct counts. 5. Calls `updateTopicAnalytics()` → classifies each topic as weak (accuracy < 50%) or strong, assigning improvement priority (high/medium/low). 6. Calls `updatePracticeStreaks()` → tracks consecutive practice days. 7. Appends to `progressHistory` (last 30 days rolling window).

Login Streaks: `updateLoginStreaks()` is invoked from the dashboard endpoint to track daily visit patterns independently from practice activity.

2.5.3 OpenAI Service (openaiService.js)

Function	Purpose
`generateStudyContent(topic, mode, syllabusContext)`	Generates structured study material (HTML content + sections + key points) using web-scraped context and AI. Uses mode-specific model selection and token budgets.
`generateQuestionsForTopic(topic, options)`	Generates MCQ or descriptive questions with Bloom's Taxonomy targeting, web context grounding, and dedup exclusion. Uses structured JSON schemas for Gemini.
`parseSyllabusContent(content)`	AI-powered extraction of topics, subtopics, estimated hours, importance, and course level.
`evaluateDescriptiveAnswer({question, modelAnswer, userAnswer})`	AI-based grading of descriptive answers. Returns score (0-10), percentage (0-100), feedback, and key-points coverage. Forces Groq for speed.
`analyzePerformance(topicPerformance)`	AI-driven analysis returning weak areas, strengths, overall assessment, and study plan.

2.5.4 AI Failover Configuration (config/openai.js)



Key Activity: The `generateJSON()` wrapper adds robust JSON extraction (markdown code block stripping, bracket detection, trailing comma cleanup, LaTeX escape fixing) and multi-pass parsing with a graceful text fallback.

2.5.5 Scraper Service (scraperService.js)

Activity: Searches the Wikipedia API for the topic → fetches the top article's HTML via the Parse API → extracts text from `<p>`, `<h2>`, `<h3>`, `<h4>` tags using Cheerio → truncates to 10,000 characters → returns as contextual grounding.

2.5.6 Utilities

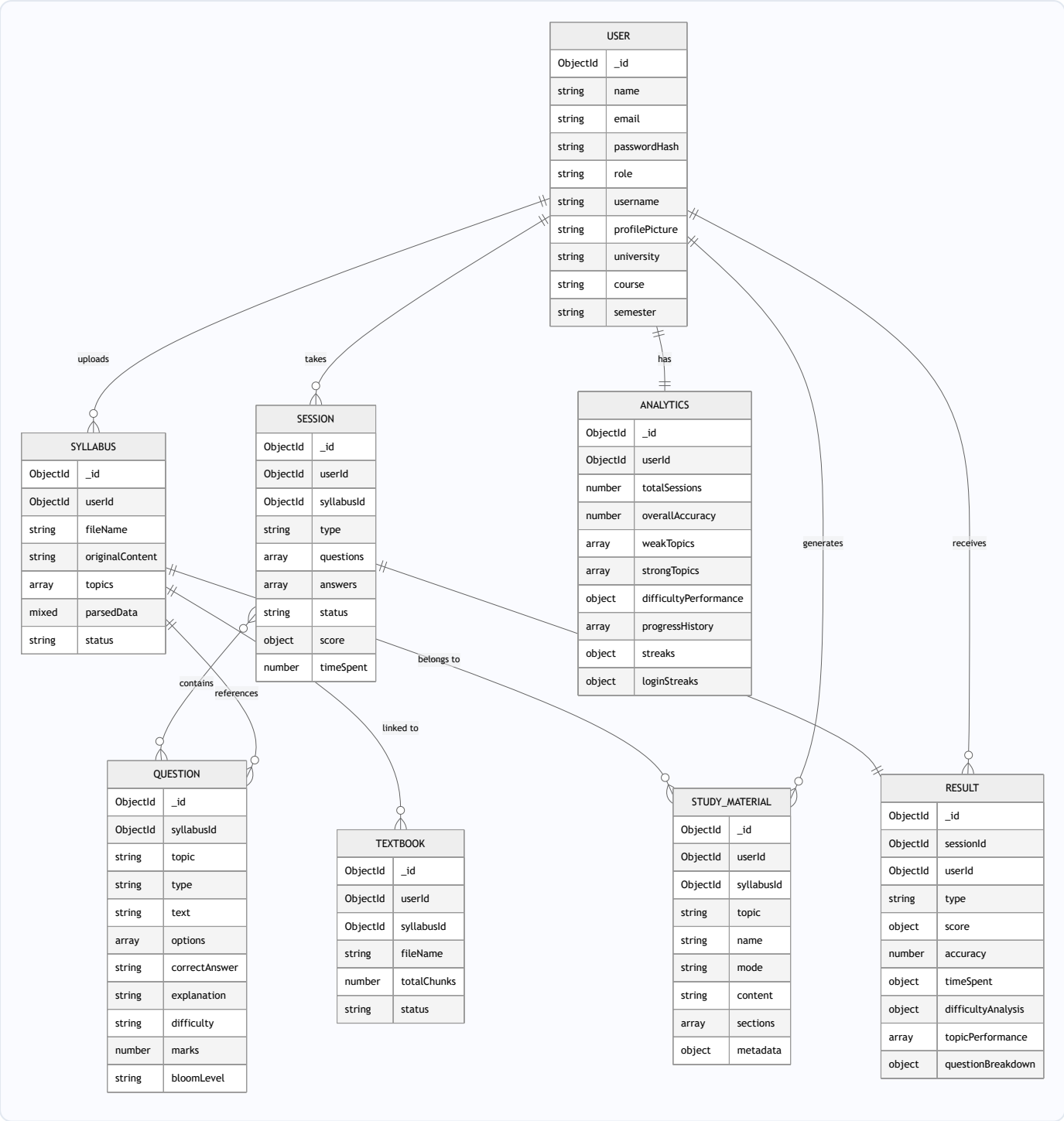
Utility	Responsibility
Topic Extractor	AI-first topic extraction from syllabus text, with regex-based fallback (detects Markdown headers, unit/chapter patterns, numbered lists, bullet points)
Bloom's Taxonomy	Defines 6 cognitive levels with action verbs, question stems for MCQ/descriptive, and weighted distribution functions
Difficulty Classifier	Classifies question difficulty based on structural features
PDF Parser	Extracts text from PDF buffers using a worker-based approach

---

2.6 LAYER 6 — Data & External Services Layer

2.6.1 Database (MongoDB Atlas via Mongoose)

Entity-Relationship Model:

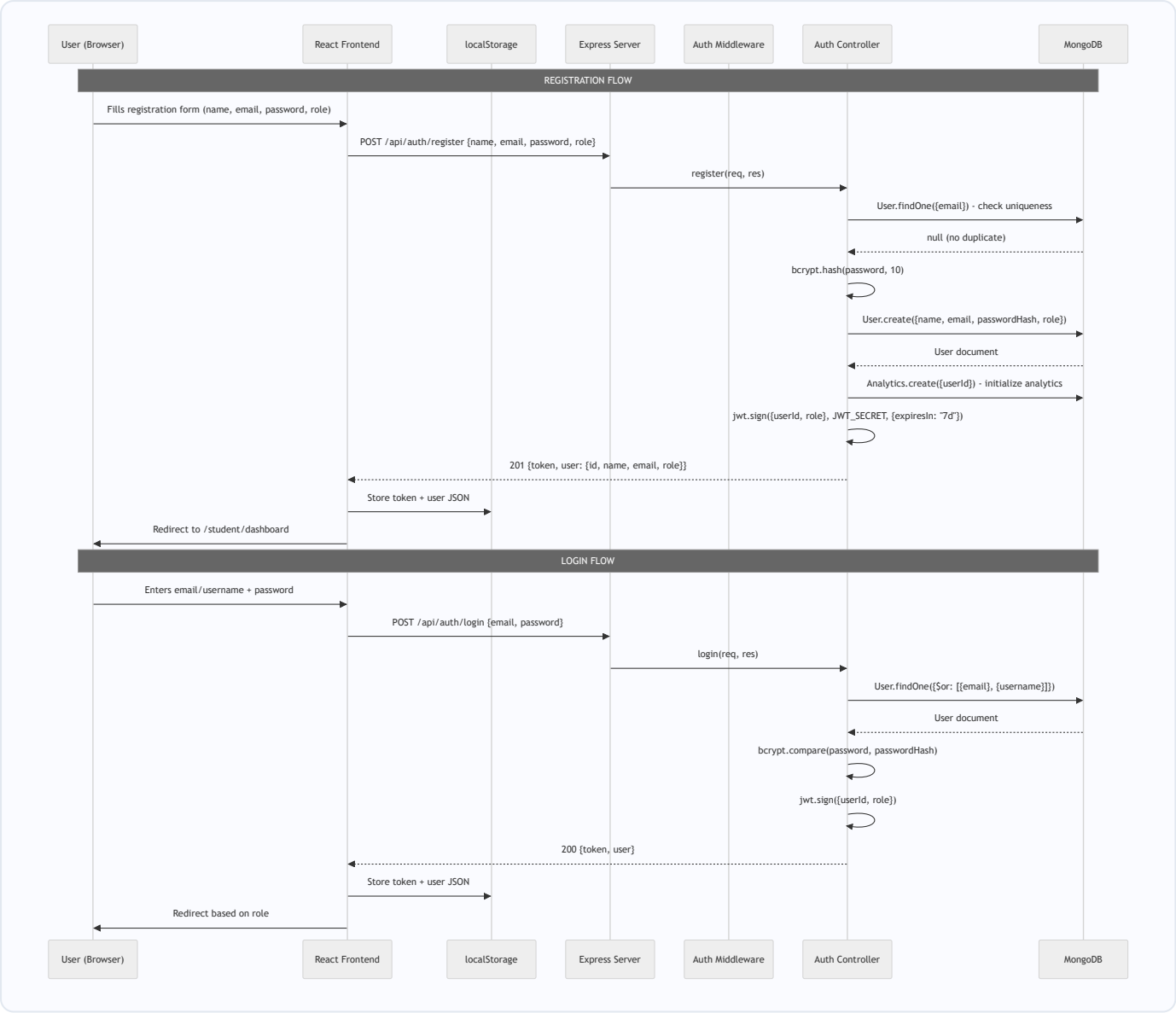


2.6.2 External APIs

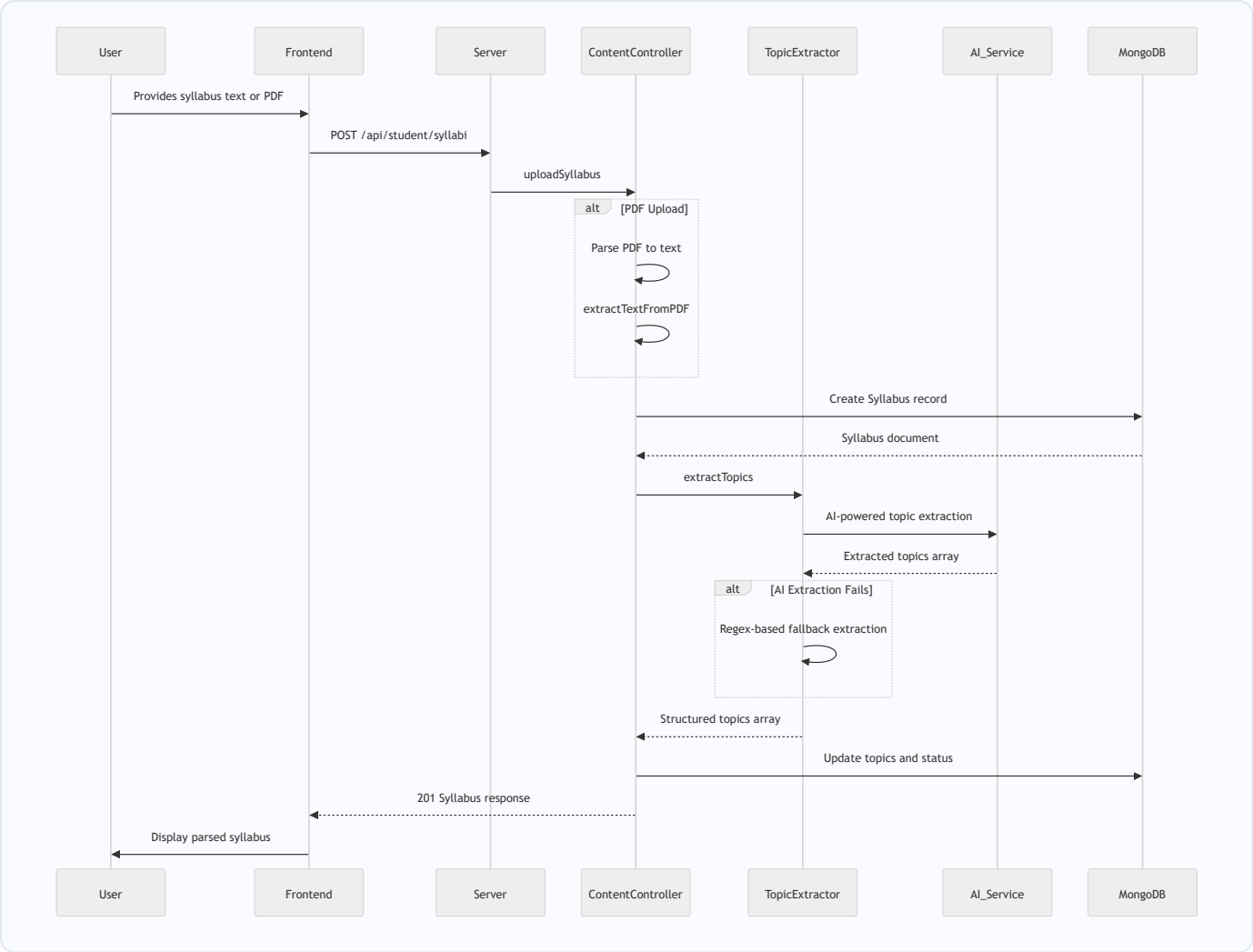
Service	Protocol	Purpose
Groq (LLaMA 3.1-8B, LLaMA 3.3-70B)	HTTPS REST	Primary LLM for content generation, question generation, answer evaluation
Google Gemini (2.5 Flash, 2.0 Pro)	HTTPS REST	Secondary/fallback LLM with structured JSON output via schema
Wikipedia API	HTTPS REST	Web scraping for factual grounding of AI-generated content
Gmail SMTP (Nodemailer)	SMTP	Password reset email delivery

3. End-to-End Data Flow Narratives

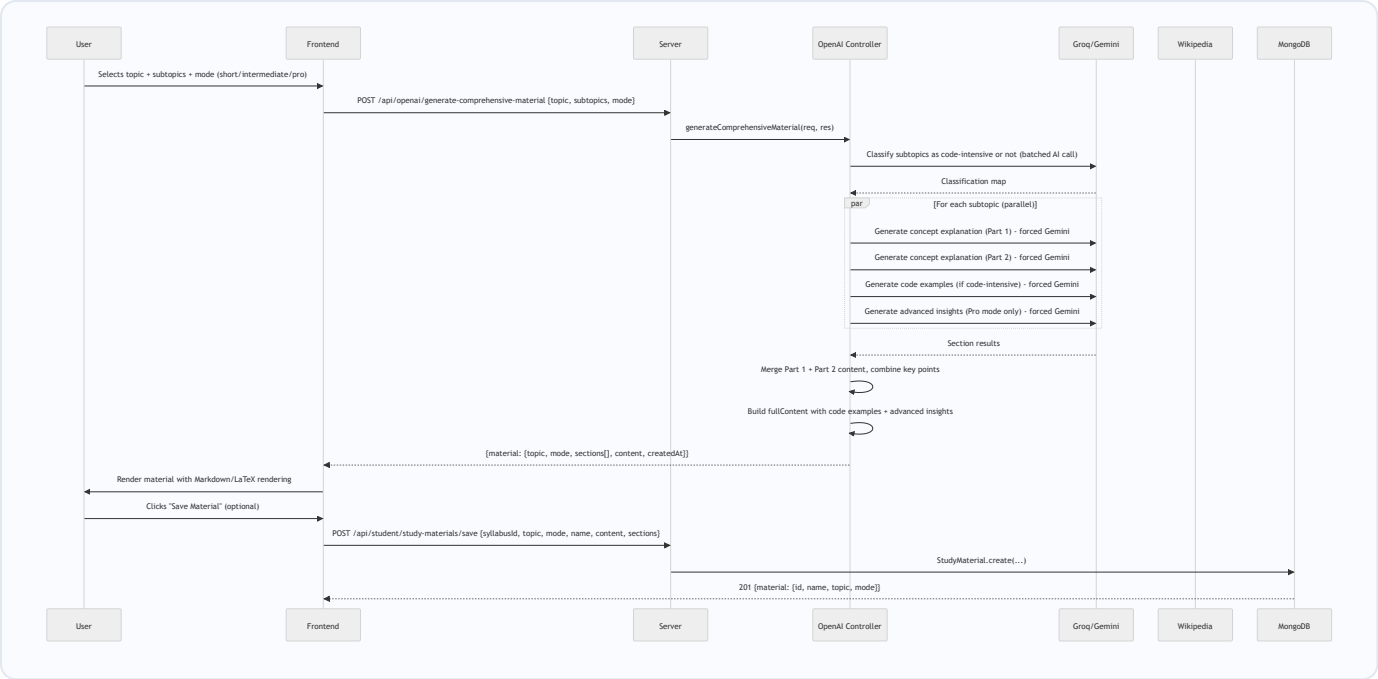
3.1 User Registration & Login Flow



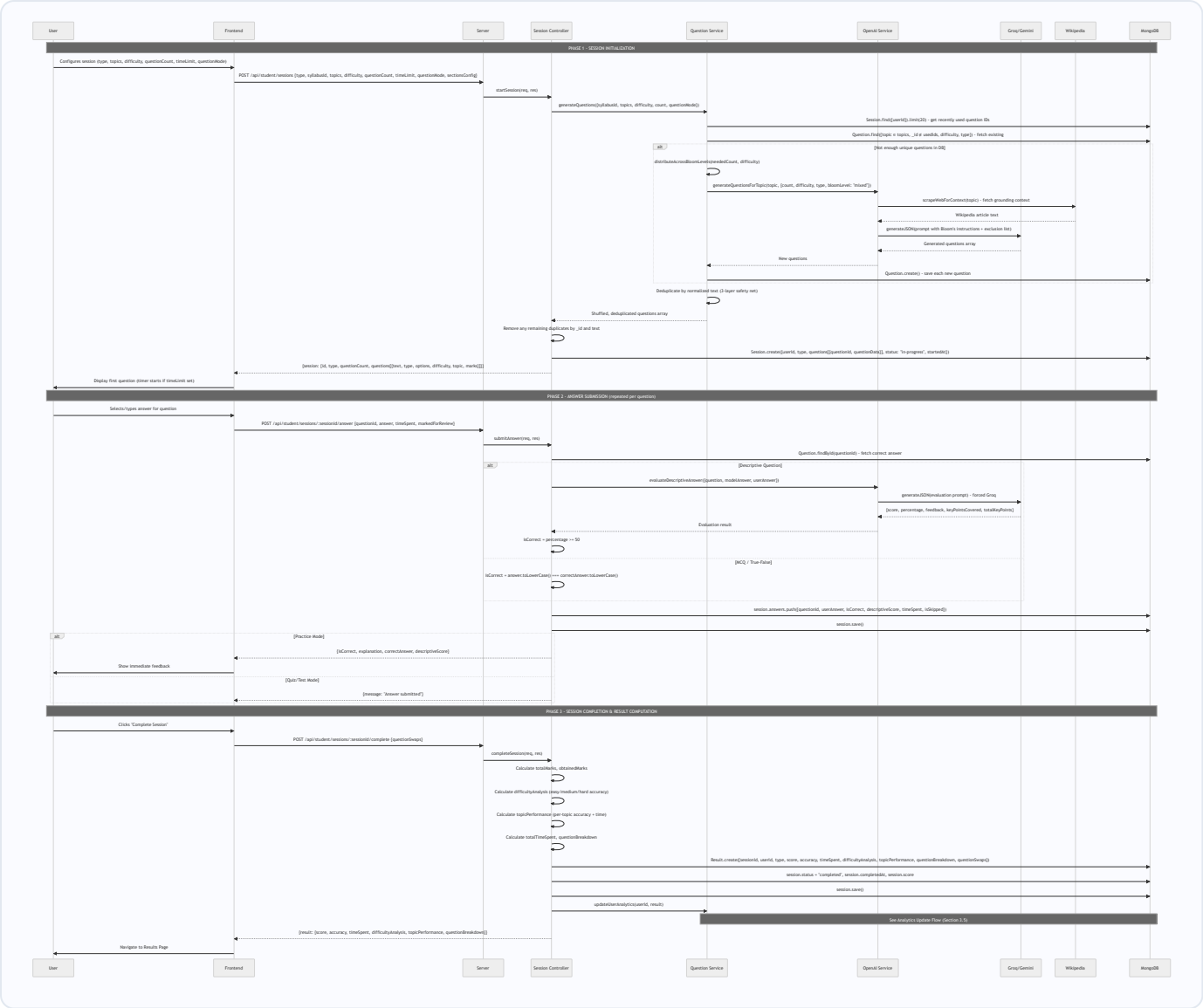
3.2 Syllabus Upload & Topic Extraction Flow



3.3 Study Material Generation Flow



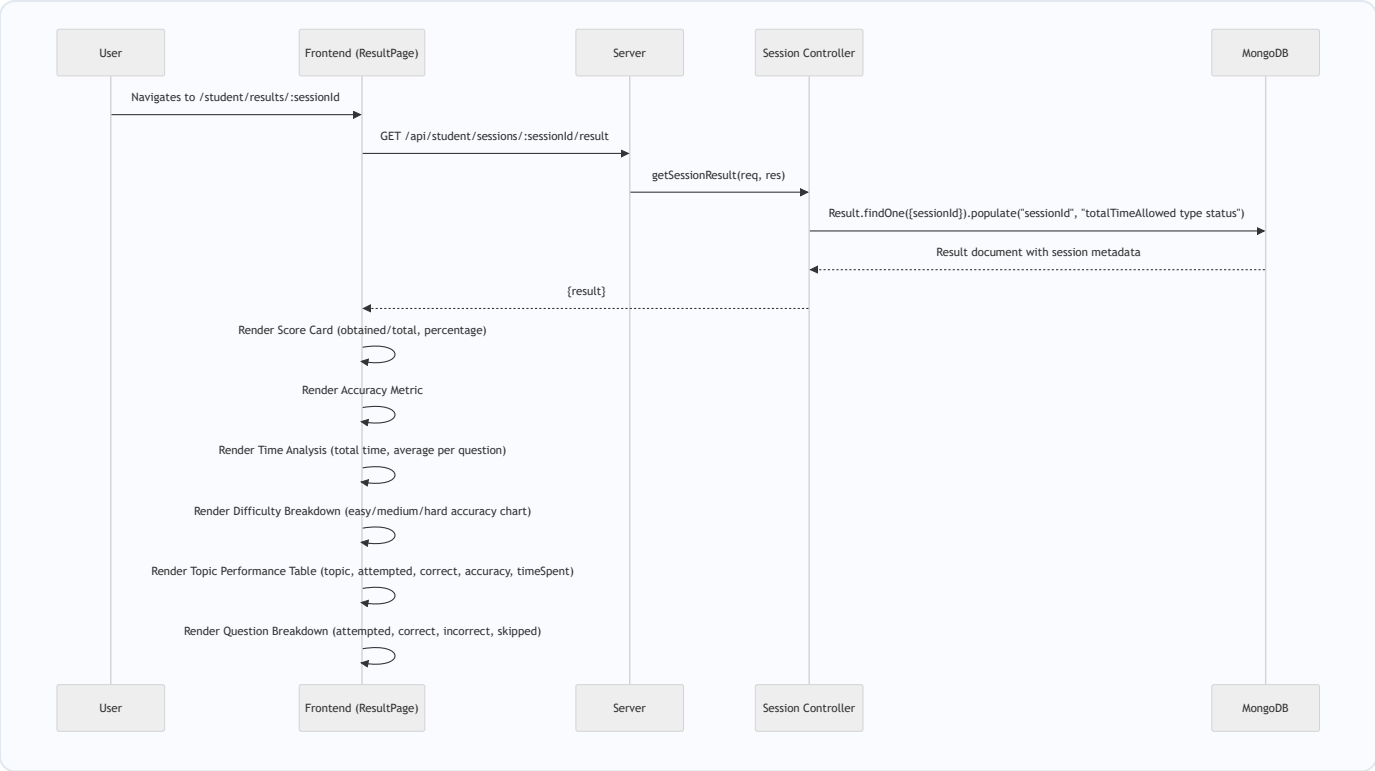
3.4 Test/Quiz/Practice Session — Complete Lifecycle



3.5 Result Analysis — Question Analysis & Time Analysis

When the user views the **Result Page**, the following data is retrieved and presented:

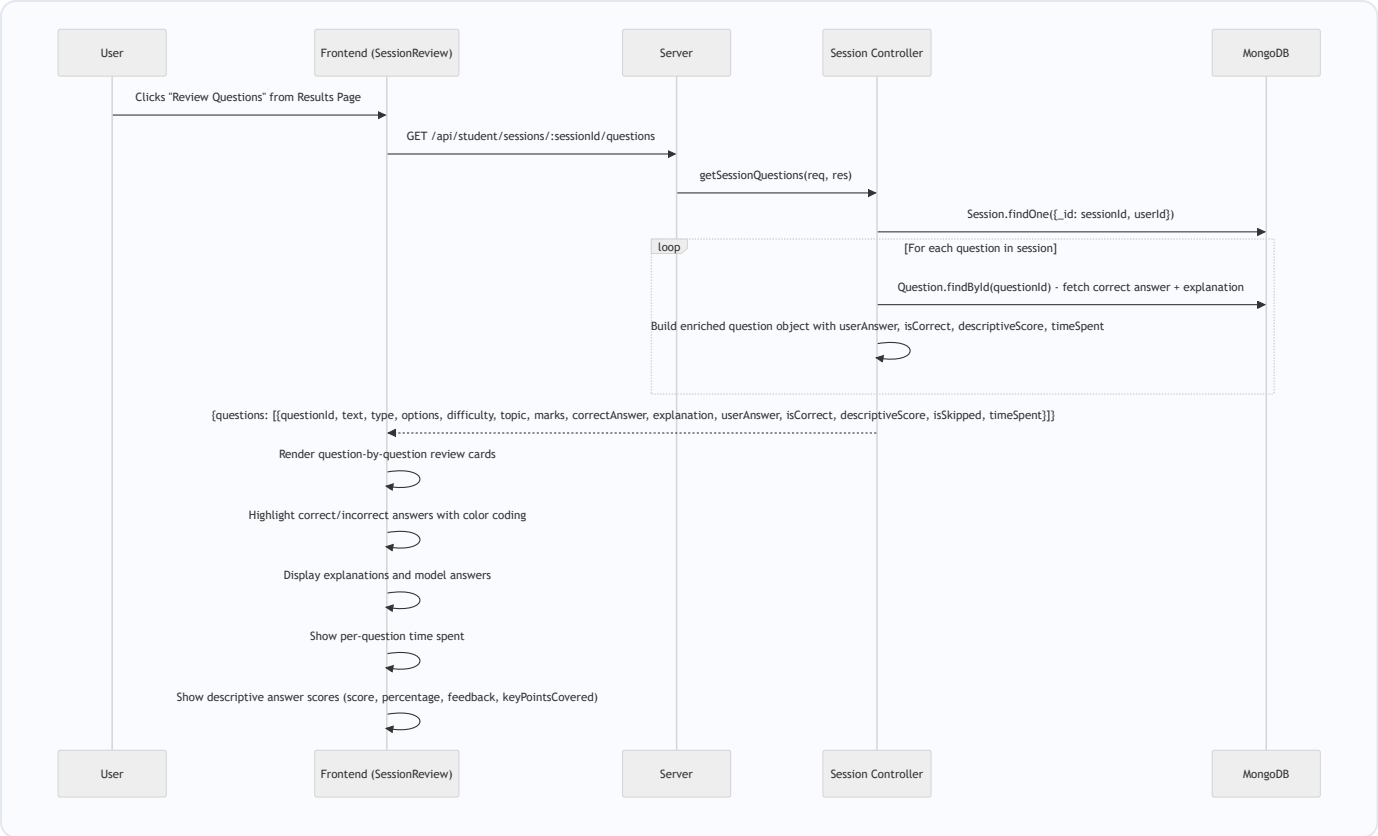
3.5.1 Result Page Data Flow



Result Object Structure:

Field	Analysis Type	Content
`score.obtained / score.total / score.percentage`	**Score Analysis**	Raw marks obtained, total possible, percentage
`accuracy`	**Accuracy Analysis**	Percentage of correct answers out of attempted
`timeSpent.total / timeSpent.average`	**Time Analysis**	Total seconds spent, average seconds per question
`difficultyAnalysis.easy/medium/hard`	**Difficulty Analysis**	Per-difficulty: attempted, correct, accuracy percentage
`topicPerformance[]`	**Topic-wise Analysis**	Per-topic: attempted, correct, accuracy, time spent
`questionBreakdown`	**Question Analysis**	Total, attempted, correct, incorrect, skipped counts
`questionSwaps[]`	**Behaviour Analysis**	Questions the user swapped/navigated to, with timestamp and time spent

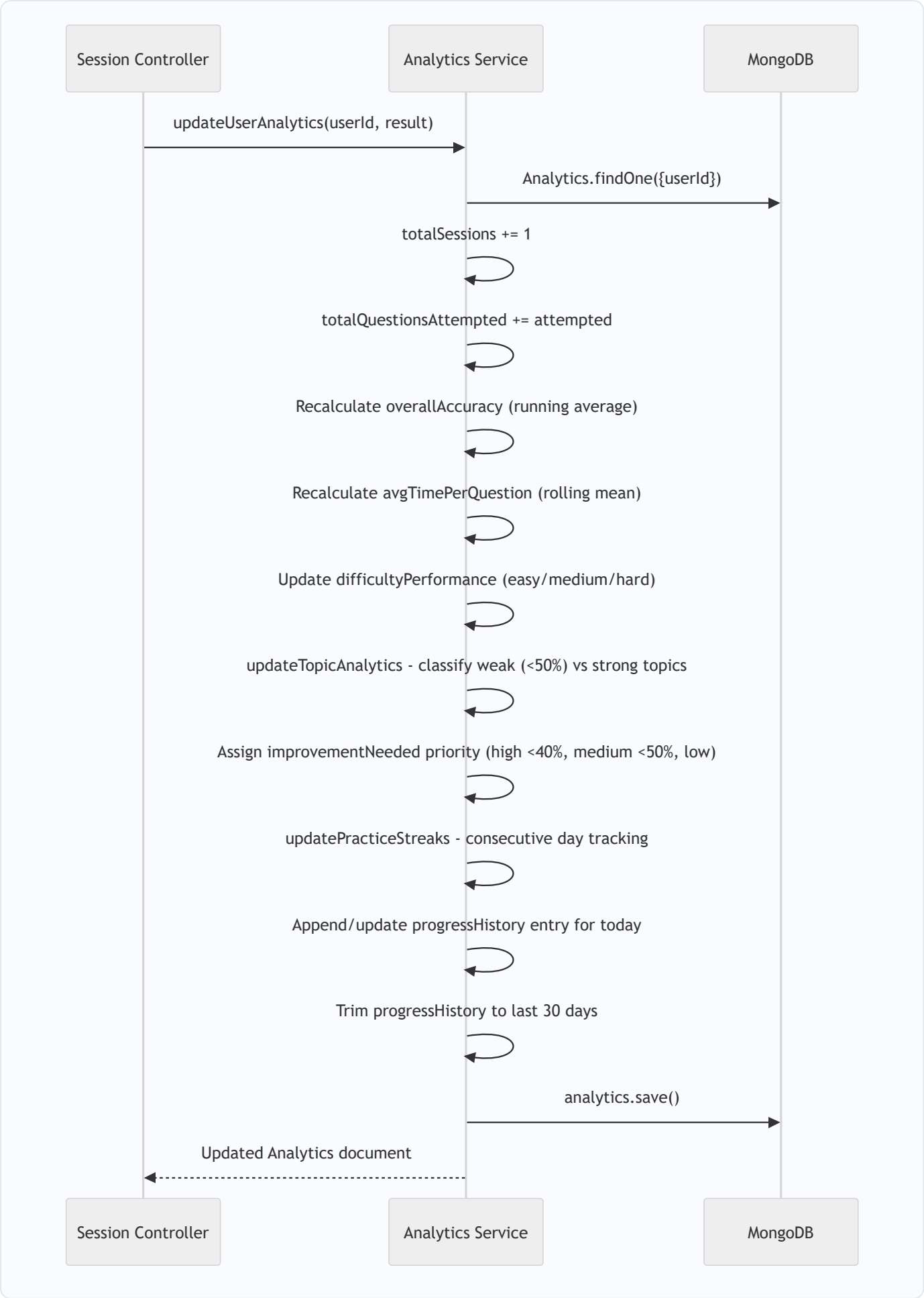
3.5.2 Session Review — Detailed Question Analysis



Per-Question Analysis Data:

Data Point	Description
`userAnswer`	The student's submitted answer
`correctAnswer`	The reference/model answer
`isCorrect`	Boolean correctness flag
`explanation`	Detailed explanation of the correct answer
`timeSpent`	Seconds spent on this specific question
`descriptiveScore.score`	0-10 AI-evaluated score (descriptive questions only)
`descriptiveScore.percentage`	0-100 percentage equivalent
`descriptiveScore.feedback`	AI-generated 1-2 sentence feedback
`descriptiveScore.keyPointsCovered / totalKeyPoints`	Coverage metric
`difficulty`	Easy/Medium/Hard classification
`topic`	The syllabus topic the question belongs to
`bloomLevel`	Bloom's Taxonomy cognitive level

3.6 Cumulative Analytics Update Flow


Session Controller
Analytics Service
MongoDB
updateUserAnalytics(userId, result)
Analytics.findOne({userId})
totalSessions += 1
totalQuestionsAttempted += attempted
Recalculate overallAccuracy (running average)
Recalculate avgTimePerQuestion (rolling mean)
Update difficultyPerformance (easy/medium/hard)
updateTopicAnalytics - classify weak (<50%) vs strong topics
Assign improvementNeeded priority (high <40%, medium <50%, low)
updatePracticeStreaks - consecutive day tracking
Append/update progressHistory entry for today
Trim progressHistory to last 30 days
analytics.save()
Updated Analytics document
Session Controller
Analytics Service
MongoDB

4. Summary of Key Activities per Layer

Layer	Key Activities
Presentation	Form rendering, route guarding, state-driven UI updates, Markdown/LaTeX rendering, real-time timer management during sessions
Client Service	JWT injection, API abstraction, Redux async thunks, localStorage persistence, error interception with auto-logout
API Gateway	Request parsing, CORS enforcement, JWT verification, role-based authorization, 404/500 error standardization
Business Logic	Input validation, password hashing, token management, session lifecycle orchestration, score/analytics computation, ownership verification
Service/Utilities	AI prompt engineering, multi-model failover with retry, web scraping for grounding, question deduplication, Bloom's Taxonomy distribution, streak tracking, email dispatch, PDF parsing, topic extraction
Data/External	MongoDB CRUD + aggregation, Groq/Gemini LLM inference, Wikipedia context retrieval, Gmail SMTP delivery, startup duplicate cleanup

Note: The Teacher module routes exist as placeholder stubs (returning "Coming soon" responses) and are designated for Phase 2 development. The RAG (Retrieval-Augmented Generation) module with Pinecone vector storage is architecturally integrated (model: `Textbook` , service: `ragService` , controller: `ragController`) but is currently **detached** (commented out) from active routes.